

# Uniswap Labs The Compact Emissary Audit



**Uniswap**

**September 19, 2025**

# Table of Contents

|                                                                                                             |    |
|-------------------------------------------------------------------------------------------------------------|----|
| Table of Contents                                                                                           | 2  |
| Summary                                                                                                     | 3  |
| Scope                                                                                                       | 4  |
| System Overview                                                                                             | 5  |
| Medium Severity                                                                                             | 6  |
| M-01 Repeated Scheduling of Key Removal May Block Execution                                                 | 6  |
| M-02 Key Removal Fails When More Than 256 Keys Are Registered                                               | 6  |
| Low Severity                                                                                                | 7  |
| L-01 Different Pragma Directives                                                                            | 7  |
| L-02 Incorrect Input Validation in <code>_registerMultisig</code>                                           | 7  |
| L-03 Mismatch Between Authorization of Keyhashes and Multisigs                                              | 8  |
| L-04 Index Overflow Causes Incorrect Key Indexing                                                           | 8  |
| Notes & Additional Information                                                                              | 9  |
| N-01 Insufficient P-256 Point Validation In <code>isValidKey</code>                                         | 9  |
| N-02 Magic Numbers                                                                                          | 9  |
| N-03 Custom Errors in <code>require</code> Statements                                                       | 10 |
| N-04 Inconsistent Order Within Contracts                                                                    | 10 |
| N-05 Missing Security Contact                                                                               | 11 |
| N-06 Incorrect Index Reported in <code>MultisigSignerIndexOutOfRange</code>                                 | 12 |
| N-07 Removing a Non-Existent Key Reverts with Incorrect Error                                               | 12 |
| N-08 The <code>canRemoveKey</code> Helper May Return <code>true</code> While <code>removeKey</code> Reverts | 12 |
| N-09 Unexpected Panic While Decoding <code>Secp256k1</code> Key Type                                        | 13 |
| Conclusion                                                                                                  | 14 |

# Summary

|           |                                  |                                |                  |
|-----------|----------------------------------|--------------------------------|------------------|
| Type      | DeFi                             | Total Issues                   | 15 (14 resolved) |
| Timeline  | From 2025-08-11<br>To 2025-09-05 | Critical Severity Issues       | 0 (0 resolved)   |
| Languages | Solidity                         | High Severity Issues           | 0 (0 resolved)   |
|           |                                  | Medium Severity Issues         | 2 (2 resolved)   |
|           |                                  | Low Severity Issues            | 4 (4 resolved)   |
|           |                                  | Notes & Additional Information | 9 (8 resolved)   |

# Scope

OpenZeppelin audited the [Uniswap/emissary](#) repository at commit [715a4d7](#).

In scope were the following files:

```
src
├─ BaseKeyVerifier.sol
├─ GenericKeyManager.sol
├─ KeyLib.sol
├─ KeyManagerEmissary.sol
├─ interfaces
│   └─ IERC1271.sol
│   └─ ISignatureVerifier.sol
└─ types
    └─ VerificationContext.sol
```

# System Overview

The Emissary project is an on-chain registry designed to facilitate secure and flexible delegation of signature authority. It supports both secp256k1 and P-256 cryptographic key formats, accommodating Hardware Security Module (HSM) and WebAuthn implementations. This functionality acts as a fallback mechanism for signature validation, enhancing the robustness of decentralized applications (dApps) and smart contracts.

Emissary's architecture enables the delegation of signing authority, allowing for more adaptable and secure interactions within the ecosystem and other decentralized platforms. By integrating with various cryptographic standards and hardware-based solutions, it ensures that signature validation remains reliable even when primary verification methods are unavailable.

The Emissary is part of The Compact, an ownerless ERC-6909 contract that enables the voluntary creation and mediation of reusable resource locks. It allows tokens to be credibly committed for spending in exchange for performing actions across arbitrary, asynchronous environments, and to be claimed once the specified conditions are met. Its role in the system is to provide a fallback verification mechanism for sponsors when authorizing claims. This is particularly useful for:

- smart contract accounts that may update their EIP-1271 (contract-based signature verification) logic
- accounts using EIP-7702 delegation that relies on EIP-1271
- situations where the sponsor delegates claim verification to a trusted third party

Refer to the [original documentation](#) and [The Compact documentation](#) for additional details.

# Medium Severity

## M-01 Repeated Scheduling of Key Removal May Block Execution

The `scheduleKeyRemoval` and `scheduleMultisigRemoval` functions define a delay period after which the associated `remove*` functions can be executed. These scheduling functions are accessible to accounts authorized via `_checkKeyManagementAuthorization`, which may be overridden in child contracts. They also verify that the specified key or multisig is currently registered before scheduling its removal. The intended design ensures that removals are scheduled and executed only by authorized entities after a defined delay period.

Currently, these functions do not verify whether a key or multisig is already pending removal. This omission allows multiple authorized accounts to repeatedly reschedule the same key or multisig for removal. Each new scheduling call resets the removal delay, which can indefinitely postpone the actual removal. Consequently, other authorized entities attempting to complete the removal may be consistently prevented from doing so.

Consider adding a validation step to ensure that a key or multisig already scheduled for removal cannot be scheduled again.

**Update:** Resolved in [pull request #2](#).

## M-02 Key Removal Fails When More Than 256 Keys Are Registered

The `GenericKeyManager` contract stores per-account keys in `keyHashes[account]` and uses a 256-bit bitmap in each multisig to mark the key indices that are signers. When removing a key that is not the last array element, the contract applies a swap-and-pop algorithm: it moves the last key into the removed slot and then updates all multisigs that reference the moved index. A range check at `_removeKey` ensures that both the original index and the moved key's old index are within 0–255 to fit the bitmap.

However, when the array length exceeds 256, `oldIndex = accountKeyHashes.length - 1` is  $\geq 256$ . Any attempt to remove a non-last key must update the moved key's old position,

so the range check reverts. As a result, only the last key can be removed, and earlier keys become effectively irremovable. Since key registration is unrestricted, any user can push an account over 256 keys, after which most removals revert. There is no warning or limit in `registerKey`, so this can occur unintentionally.

Consider enforcing a hard cap of 256 keys per account in `registerKey` and reverting with a clear error once the limit is reached. In addition, consider documenting the capacity.

**Update:** Resolved in [pull request #5](#).

# Low Severity

## L-01 Different Pragma Directives

In order to clearly identify the Solidity version with which the contracts will be compiled, pragma directives should be fixed and consistent across file imports.

Throughout the codebase, different pragma directives are being used:

- `BaseKeyVerifier.sol` has the `pragma solidity ^0.8.30;` pragma directive and imports `BaseKeyVerifier.sol`, which has a different pragma directive.
- `GenericKeyManager.sol` has the `pragma solidity ^0.8.30;` pragma directive and imports `GenericKeyManager.sol`, which has a different pragma directive.
- `KeyLib.sol` has the `pragma solidity ^0.8.27;` pragma directive and imports `KeyLib.sol`, which has a different pragma directive.
- `KeyManagerEmissary.sol` has the `pragma solidity ^0.8.27;` pragma directive and imports `KeyManagerEmissary.sol`, which has a different pragma directive.

Consider using the same, fixed pragma directive across all files.

**Update:** Resolved in [pull request #3](#).

## L-02 Incorrect Input Validation in `_registerMultisig`

Within the `_registerMultisig` function, the number of signers (`signerCount`) is determined by taking the length of `signerIndices` and casting it to `uint8`. A subsequent

check ensures that `signerCount` is greater than 0 and less than or equal to 255. This validation order is incorrect because the cast to `uint8` may truncate the original length before the range check is applied, potentially bypassing proper validation.

Consider validating the length of `signerIndices` directly against the intended bounds before casting to `uint8`.

**Update:** Resolved in [pull request #4](#).

## L-03 Mismatch Between Authorization of Keyhashes and Multisigs

Within the `_registerKey` function, authorization is verified using the `_checkKeyManagementAuthorization` function. This logic is triggered for both the `_registerKey` function, which registers the key for a given account, and the `register` function, which registers the key for the caller. However, the issue is that when registering a multisig, authorization is only checked if the registration is [performed for an account](#), but [not if it is performed by the caller](#).

Consider unifying the behavior to either verify authorization only when the call is made for the account, or verifying it across all paths.

**Update:** Resolved in [pull request #6](#).

## L-04 Index Overflow Causes Incorrect Key Indexing

`Key.index` is a `uint16`, 1-based index of an array where 0 denotes "unregistered". During registration, the implementation appends the new key hash to `keyHashes[account]` and then sets `key.index = uint16(keyHashes[account].length)`. During removal, the code relies on this stored index to compute a 0-based array position (`uint256(key.index) - 1`) and performs swap-and-pop on `keyHashes[account]`. Multisig signer bitmaps are also updated based on these array positions and include a separate check ensuring that the indices used in the bitmap are  $< 256$ .

The `cast to uint16` can overflow once `keyHashes[account].length >= 65,536`. At length 65,536, the cast produces 0 and at 65,537, it produces 1, and so on. An index of 0 causes a revert in `_removeKey` when computing `key.index - 1`. A wrapped non-zero index can cause `_removeKey` to operate on the wrong array slot: the function may swap and



pop a different element, update multisig bitmaps for the wrong key, and leave the array and mapping out of sync (e.g., delete `keys[account][keyHash]` while not actually removing `keyHash` from `keyHashes[account]`).

Consider adding an explicit check before casting to ensure that the array length does not exceed `type(uint16).max`.

**Update:** Resolved in [pull request #5](#). The team stated:

| *Implicitly addressed by fix for M-02.*

# Notes & Additional Information

## N-01 Insufficient P-256 Point Validation In `isValidKey`

The `isValidKey` function supports multiple key types (Ethereum address derived from a secp256k1 public key, P-256 public keys, and WebAuthn P-256 public keys). For P-256, it does not verify that the public key coordinates fall within the prime field range  $[0, p - 1]$  or that the point satisfies the curve equation. This means that malformed or off-curve points may be accepted. Since P-256 has a cofactor of 1, there are no (small) subgroups, beside the trivial one. Therefore, once the point is confirmed to be on the curve, there is no need for an additional check verifying that it belongs to the correct subgroup.

Consider enforcing proper validation for P-256 keys: check coordinate ranges, reject the point at infinity, and confirm the curve equation holds.

**Update:** Resolved in [pull request #7](#), commits [73d4c33](#) and [0873812](#).

## N-02 Magic Numbers

Within `GenericKeyManager.sol`, multiple instances of literal values with unexplained meanings were identified.

Literal values such as `256` and `255` appear in multiple places in the contract ([1](#), [2](#), [3](#), [4](#)). Without clear definitions, it is not immediately clear what these values represent or why they are used.

Consider defining and using `constant` variables instead of using literals to improve the readability of the codebase.

**Update:** Resolved in [pull request #7](#) at commit [c8628ab](#).

## N-03 Custom Errors in `require` Statements

Since Solidity [version 0.8.26](#), custom error support has been added to `require` statements. Initially, this feature was only available through the IR pipeline. However, Solidity [0.8.27](#) extended its support to the legacy pipeline as well.

`GenericKeyManager.sol` contains multiple instances of `if-revert` statements that could be replaced with `require` statements:

- The `if (usageCount > 0) { revert KeyStillInUse(keyHash, usageCount); }` statement
- The `if (newIndex >= 256 || oldIndex >= 256) { revert MultisigSignerIndexOutOfRange(newIndex); }` statement
- The `if ((cfg.signerBitmap & (1 << newIndex)) != 0) { revert MultisigSignerIndexCollision(msHash, newIndex); }` statement
- The `if (index >= 256) revert MultisigSignerIndexOutOfRange(index)` statement

For conciseness and gas savings, consider replacing `if-revert` statements with `require` statements.

**Update:** Resolved in [pull request #7](#) at commit [7dbcf6f](#).

## N-04 Inconsistent Order Within Contracts

Throughout the codebase, multiple instances of contracts having an inconsistent order of functions were identified:

- The `BaseKeyVerifier` contract in `BaseKeyVerifier.sol`
- The `GenericKeyManager` contract in `GenericKeyManager.sol`
- The `KeyManagerEmissary` contract in `KeyManagerEmissary.sol`

To improve the project's overall legibility, consider standardizing ordering throughout the codebase as recommended by the [Solidity Style Guide \(Order of Functions\)](#).

**Update:** Acknowledged, not resolved. The team stated:

| Acknowledged, won't fix

## N-05 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Throughout the codebase, multiple instances of contracts missing a security contact were identified:

- The [BaseKeyVerifier contract](#)
- The [GenericKeyManager contract](#)
- The [KeyLib library](#)
- The [KeyManagerEmissary contract](#)
- The [IERC1271 interface](#)
- The [ISignatureVerifier interface](#)

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

**Update:** Resolved in [pull request #7](#) at commit [ea9364b](#).

## N-06 Incorrect Index Reported in MultisigSignerIndexOutOfRange

In `_removeKey`, both `newIndex` and `oldIndex` are checked against the 0–255 range. However, the revert always reports `newIndex`, even when `oldIndex` is the value out of range. This results in errors showing a valid index, making the root cause unclear.

Consider reverting with the actual failing index (e.g., separate checks for `newIndex` and `oldIndex`).

**Update:** Resolved in [pull request #7](#) at commit [9e080b0](#).

## N-07 Removing a Non-Existent Key Reverts with Incorrect Error

In the `GenericKeyManager` contract, key removal requires a timelock before deletion. The `_removeKey` function first checks whether the key's `removalTimestamp` is set and has expired. For an unregistered key, this value is zero, which triggers a revert with `KeyRemovalUnavailable(0)` instead of signaling that the key does not exist.

Consider validating that the key is registered before performing the timelock check, reverting with `KeyNotRegistered` when appropriate.

**Update:** Resolved in [pull request #7](#) at commit [7551939](#).

## N-08 The `canRemoveKey` Helper May Return `true` While `removeKey` Reverts

The `canRemoveKey` helper is meant to indicate whether a key can be removed. It currently only checks whether the removal timelock has expired. However, the actual removal process also requires that the key is not referenced by any multisig. Since this additional condition is ignored, `canRemoveKey` may return `true` even though `removeKey` would revert with `KeyStillInUse`. This discrepancy can mislead integrators who rely on `canRemoveKey` to determine if removal is possible.

Consider updating `canRemoveKey` to reflect all conditions enforced during key removal, including multisig references, so that its result is consistent with the behavior of `removeKey`.

**Update:** Resolved in [pull request #7](#) at commit [d44a980](#).

## N-09 Unexpected Panic While Decoding Secp256k1 Key Type

The `isValidKey` function within `KeyLib` library for `KeyType.Secp256k1` verifies that the length of `key.publicKey` is equal to 32 and then attempts to decode its value to `address`. The issue is that it panics at decoding `key.publicKey` if the upper bytes (that exceed the length of the 20-byte address) are not equal to 0.

Consider adding further validation to ensure that the upper bytes are zeroed out for the `Secp256k1` key type so that `abi.decode` will succeed.

**Update:** Resolved in [pull request #7](#) at commit [684b807](#).

# Conclusion

The audited scope provides a composable foundation for key management across different protocols, including the emissary implementation within The Compact protocol.

During the audit, several medium- and low-severity issues were identified, while no high-severity issues were reported.

The Uniswap Labs team is appreciated for their cooperation throughout the review and for providing timely and clear responses to all the inquiries made by the audit team.